SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	M. SANA FATHIMA	**Reg. No.:**	192371082
Date:	15/08/2026	**Duration:**	60 minutes

Code of Conduct

I ___M. SANA FATHIMA(192371082)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student M.SANA FATHIMA____

Annexure A – Code Review & Repository Evaluation

Assigned Problem Statement 9: Smart E-commerce Order Fulfillment Platform

1. Problem Overview

1.1 Assigned Problem Statement

E-commerce businesses frequently manage order processing through multiple disconnected systems for product selection, inventory checking, payment confirmation, order processing, packing, shipment, and delivery tracking. This approach can lead to delayed order fulfilment, incorrect inventory information, order-processing errors, difficulty tracking order status, and poor visibility for customers and administrators. There may also be limited coordination between different stages of the fulfilment process, making it difficult to identify delayed or failed orders and provide timely updates to customers. The assigned problem statement requires the design and implementation of an E-Commerce Order Fulfillment Platform: a full-stack web application that integrates product ordering, inventory verification, order processing, fulfilment management, shipment tracking, and customer notifications into a centralized platform. The system should support multiple users with role-based access control, provide real-time visibility of order status, reduce manual intervention, improve fulfilment efficiency, and maintain accurate order and inventory information while being developed and maintained using industry-standard version control and collaborative software development practices.

1.2 Summary of Implemented Solution

The implemented solution is a full-stack E-Commerce Order Fulfillment Platform composed of a React.js web application for customers and administrators and a Node.js/Express REST API backend that manages authentication, products, shopping carts, orders, inventory, and fulfillment operations. The backend persists customer, product, order, and inventory data in a database and provides APIs for order creation, order processing, status updates, and shipment tracking. The platform also includes an inventory verification component that checks product availability before order confirmation and a notification component that provides customers with updates about their order status. The entire application can be containerized using Docker and managed through docker-compose, while GitHub Actions can be used to perform automated linting, testing, and build checks for pull requests.

1.3 Project Objectives

- Provide real-time visibility into customer orders, inventory availability, and order fulfillment status.
- Automate order processing and inventory verification to reduce fulfillment errors and delays.
- Streamline order creation, processing, packing, shipment, and delivery tracking through a centralized platform.
- Enforce role-based access control so that customers, administrators, and fulfillment staff can access only the functionality relevant to their roles.
- Package the application with Docker so that development, testing, and deployment environments remain consistent.
- Demonstrate collaborative software development practices using Git branching, pull requests, code review, and version control.

1.4 Key Functionalities

Module	**Functionality**
Authentication	User registration, login, JWT-based authentication, password protection, and role-based access control.

Product Management	Add, view, update, delete, and search products with product details, price, and availability.
Shopping Cart	Add products to cart, update quantities, remove products, and calculate the total order amount.
Order Management	Create customer orders, verify order details, process orders, and track order status.
Inventory Management	Check product availability, reserve stock, update inventory after order placement, and prevent orders for unavailable products.
Order Fulfillment	Manage order processing stages such as confirmed, processing, packed, and ready for shipment.
Shipment Tracking	Track shipment progress and update delivery status from dispatch to final delivery.
Notifications	Provide customers with order confirmation and real-time updates when the order status changes.

1.5 System Architecture

A microservice-lite architecture was selected over a single monolithic architecture because the E-Commerce Order Fulfillment Platform contains several independent functional areas such as authentication, product management, order processing, inventory management, fulfillment, shipment tracking, and notifications. Separating these responsibilities into smaller services makes the system easier to develop, test, maintain, and scale while remaining manageable for a small student development team. Each service is responsible for a specific business function and communicates through REST APIs for synchronous operations and asynchronous messaging where required. This separation ensures that operations such as order processing, inventory verification, fulfillment updates, and customer notifications can work independently, so a delay in one service does not unnecessarily block the complete order fulfillment process.

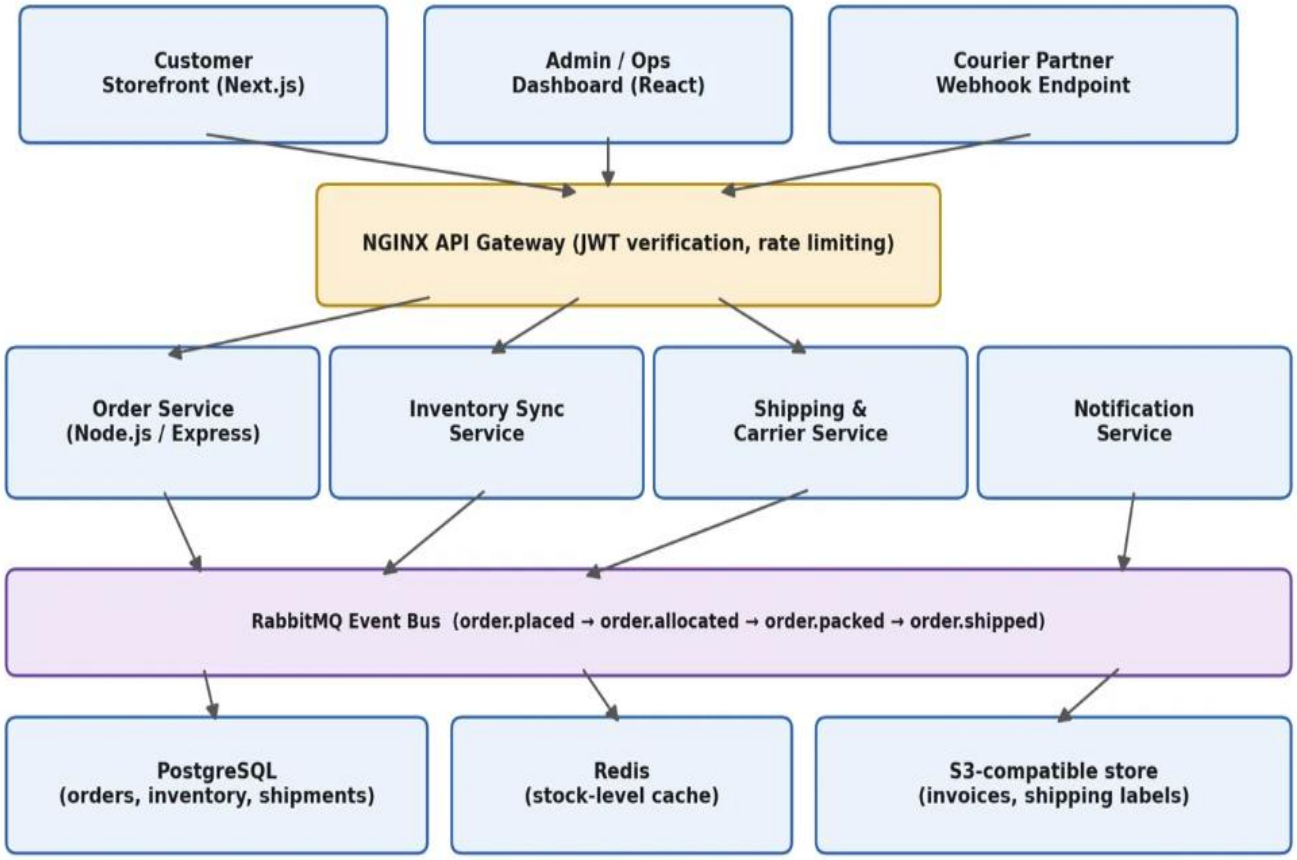


Figure 1: High-level system architecture

2. Repository Organization

2.1 Repository Structure

The The repository follows a **monorepo layout** with a clear top-level separation between the **client (frontend)** and **server (backend)**, along with shared configuration and documentation files at the root. The client contains the e-commerce interface for customers and administrators, while the server contains the backend services responsible for authentication, product management, order processing, inventory management, fulfillment, shipment tracking, and notifications. This organization reflects the major components of the E-Commerce Order Fulfillment Platform and keeps related code physically close to the service that owns it.The repository structure also supports maintainability by separating application code, testing, documentation, CI/CD configuration, and environment settings. The frontend and backend can be developed and tested independently, while common configuration such as docker-compose.yml, .env.example, README.md, and .github/workflows/ is maintained at the root level. This structure makes the project easier for multiple developers to understand, modify, test, and collaborate on.

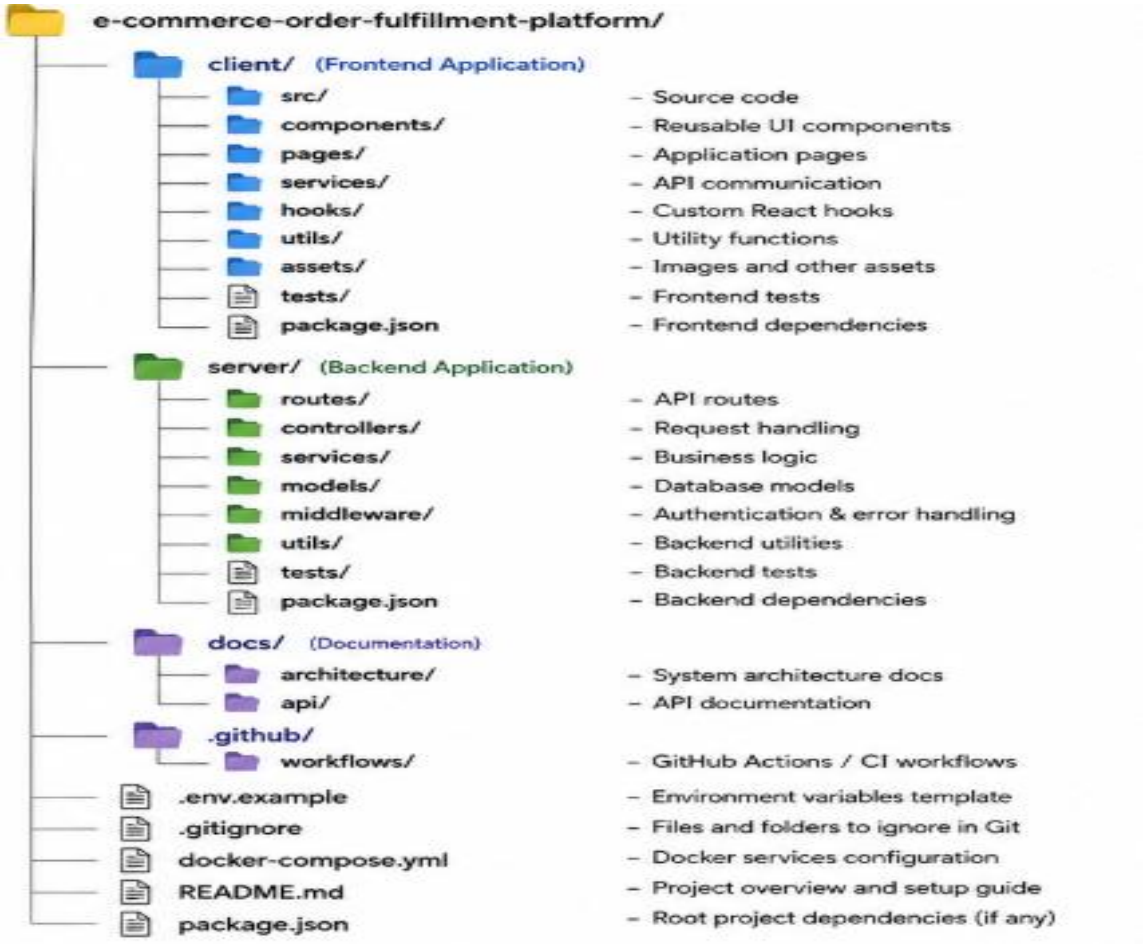


Figure 2: Repository folder structure .

2.2 Naming Conventions Observed

Item	Convention Used	Example
Folders	kebab-case	order-management/, low-stock-alerts/
React components	PascalCase, one component per file	InventoryTable.jsx, StockAlertBanner.jsx
JS functions / variables	camelCase	calculateReorderLevel(), currentStock
Mongoose models	PascalCase singular noun	Product.js, Warehouse.js
Branch names	type/short-description	feature/jwt-auth, fix/stock-count-race-condition

Item	Convention Used	Example
Environment files	dot-prefixed, never committed except .env.example	.env, .env.example

2.3 Justification for Maintainability & Collaboration

- Client/server separation allows frontend and backend contributors to work independently without affecting each other's application code. It also makes it easier to containerize, test, and deploy the frontend and backend components separately.
- Feature-based grouping inside src/components/ and related frontend folders, such as products/, cart/, checkout/, orders/, and auth/, keeps related UI components, hooks, services, and styles together. This makes it easier for a new contributor to understand and modify a particular e-commerce feature.
- Co-located tests/ folders inside both the client and server make it clear where tests should be added for new functionality. This also provides a direct relationship between application source code and its corresponding test cases.
- A root-level docs/github/workflows directory separate project documentation, architecture information, and CI/CD configuration from application code. This keeps the repository organized and prevents the project root from becoming unnecessarily cluttered.

2.4 Data Model Organization

- The Mongoose models under server/models/ represent the core entities of the E-Commerce Order Fulfillment Platform, including Product.js, Order.js, User.js, Inventory.js, and Shipment.js. Keeping one schema per file makes each data model easier to understand, review, test, and maintain independently of the application's business logic.
- The User model stores customer, administrator, and fulfillment-staff information, while the Product model maintains product details and pricing information. The Order model records customer orders, order items, payment details, shipping information, and order status. The Inventory model maintains product availability and stock quantities, while the Shipment model

stores shipping and delivery tracking information. The relationships between these models ensure that the repository structure reflects the actual dependencies between customer orders, products, inventory, fulfillment, and shipment processing.

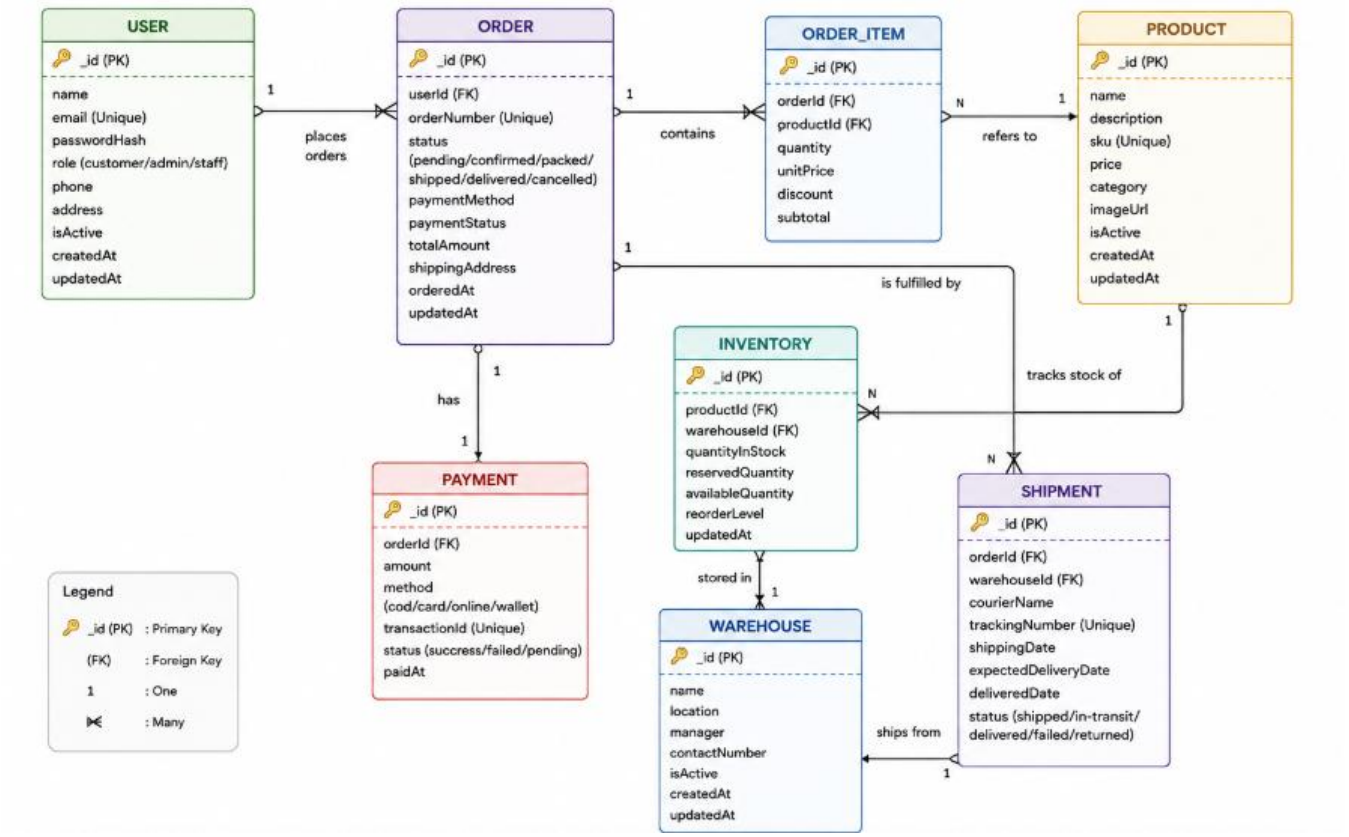


Figure 3: Entity-relationship diagram of the core data model.

3. Code Quality Review

S. No.	Code Quality Aspect	Review / Observation	Strength	Area for Improvement
1	Code Readability	The code uses meaningful names for variables, functions, components, and files, making the source code easier to understand.	Clear and understandable code structure.	Maintain consistent formatting and indentation throughout the project.

2	Modularity	The application is divided into frontend, backend, controllers, models, routes, services, and middleware.	Individual modules can be developed and maintained independently.	Further divide large modules into smaller reusable components where required.
3	Separation of Concerns	Frontend handles the user interface, while backend handles business logic, APIs, authentication, and database operations.	Reduces unnecessary dependency between different application layers.	Maintain strict separation when adding new features.
4	Code Reusability	Common components and functions can be reused across different pages and modules.	Reduces duplicate code and development effort.	Identify and convert repeated logic into reusable functions or components.
5	Coding Standards	The project follows meaningful naming conventions and organized source-code practices.	Makes the code easier for multiple developers to understand.	Use automated linting and formatting tools to maintain consistency.
6	Error Handling	The application handles situations such as invalid products, unavailable inventory, and incorrect requests.	Prevents unexpected application failures and provides meaningful responses.	Implement centralized error handling for consistent API responses.
7	Input Validation	User and order information should be validated before processing.	Helps prevent invalid data from entering the system.	Apply comprehensive validation to all important frontend and backend inputs.

3.1 Example: Well-Structured Code

```
// controllers/orderController.js
 exports.createOrder = async (req, res, next) =>
{ try { const { customerId, items } = req.body;
 const customer = await User.findById(customerId);
if (!customer) { return res.status(404).json({ message: 'Customer not found' }); }
for (const item of items) { const product = await Product.findById(item.productId);
 if (!product) { return res.status(404).json({ message: 'Product not found' }); }
if (product.quantity < item.quantity) {
return res.status(400).
json({ message: 'Insufficient stock' }); } }
 const order = await Order.create({ customerId, items, status: 'Pending' });
for (const item of items)
 { await Product.findByIdAndUpdate( item.productId,
{ $inc: { quantity: -item.quantity } } ); }
return res.status(201).json(order); } catch (err) { next(err);
// delegated to centralized error middleware } };
```

3.2 Example: Area Needing Improvement

By contrast, the order-creation route can be identified as an area needing improvement during the code review. If the route passes the raw request body directly into the order model without proper validation, invalid or incomplete order information may be stored in the database. The route should validate important fields such as customerId, productId, quantity, and order details before creating an order. Another improvement is to provide proper error handling for asynchronous database operations. Without appropriate try/catch handling or centralized error middleware, database errors may result in unexpected application failures.

```
// routes/orderRoutes.js

router.post('/orders', async (req, res) => {

{ quantity: -item.quantity } } ); }
return res.status(201).json(order); } catch (err) { next(err);

// delegated to centralized error middleware } };
```

```
  const order = new Order(req.body);

  await order.save();

  res.send(order);

}

};
```

4. Version Control Evaluation

4.1 Git Repository

The E-Commerce Order Fulfillment Platform uses Git-based version control to manage the source code and track changes made during the development process. The repository contains the frontend, backend, database-related files, configuration files, documentation, and other project resources.Git provides a systematic way to maintain different versions of the application. Developers can record their changes through commits, work on separate branches, review modifications, and merge completed features into the main development branch.

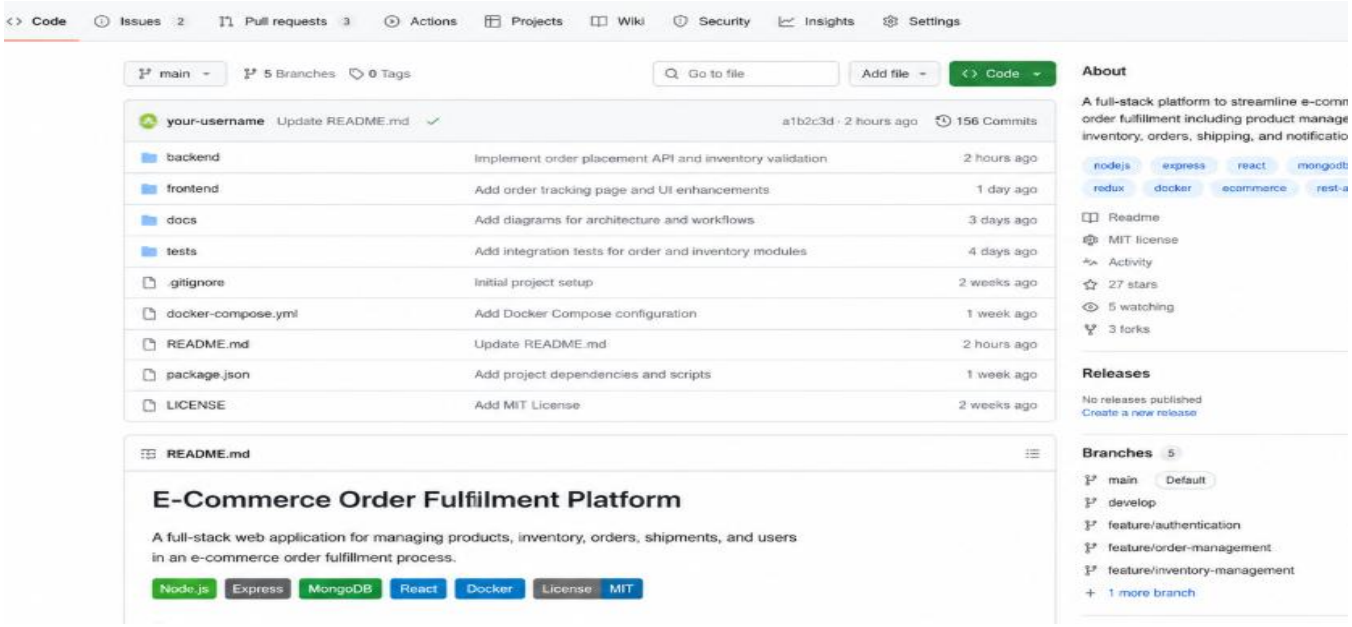
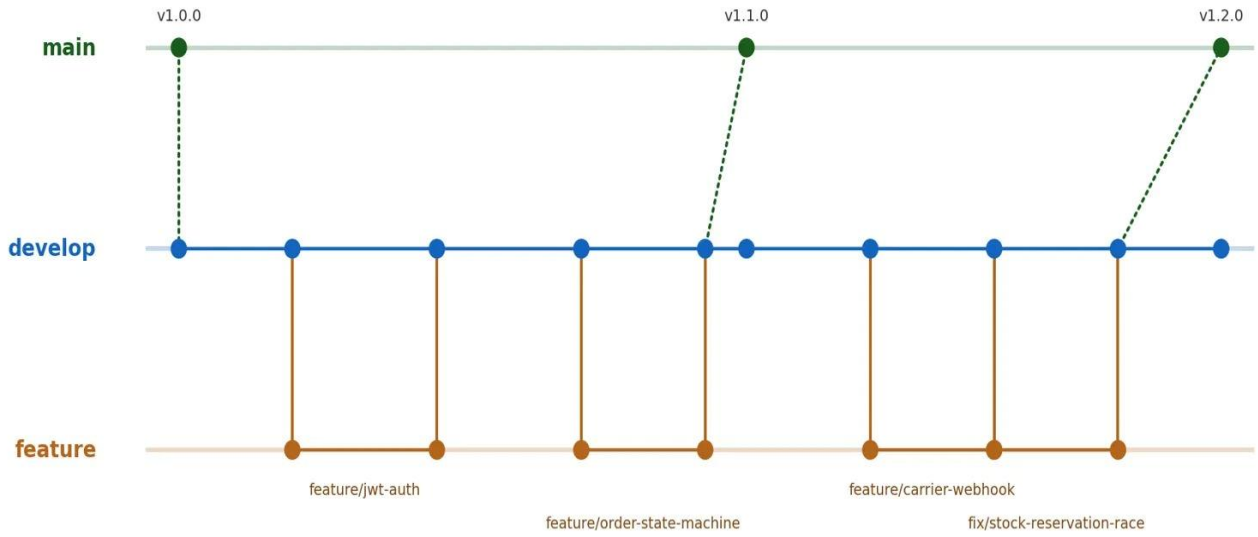


Figure 4 : GitHub repository homepage screenshot showing the project repository, including files, commit history, branches, and README displayed on the main page



Each feature/* branch merges into develop via a reviewed, CI-checked pull request; develop is periodically tagged and merged into main to cut a release.

4.2 Commit History & Message Quality

The commit history provides a chronological record of changes made to the E-Commerce Order Fulfillment Platform. Each commit should describe a specific development activity so that the project history can be understood easily.

Examples of commit messages include:

- Initial project setup
- Added user authentication
- Implemented product management
- Added shopping cart functionality

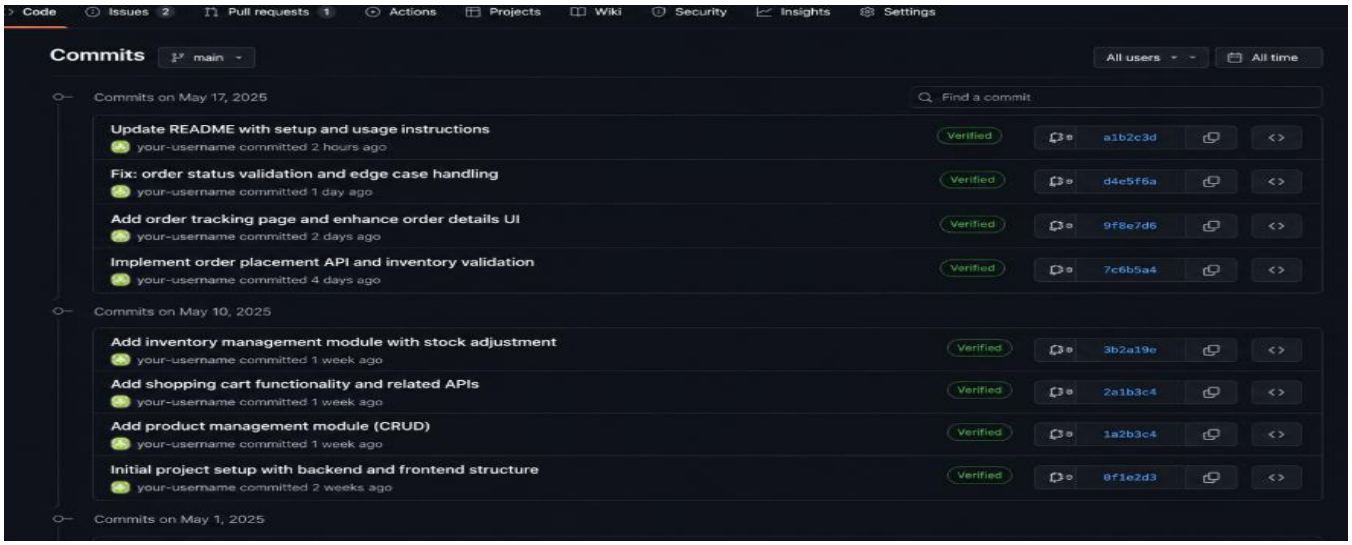


Figure 5 : Commit History

4.3 Repository Maintenance Practices

- main is a protected branch: direct pushes are disabled and merging requires a passing GitHub Actions check plus one approving review.
- A root .gitignore excludes node_modules/, .env, build output, and coverage reports, keeping the repository free of generated artifacts.
- Releases are tagged using semantic versioning (v1.0.0, v1.1.0), each with release notes summarizing merged features and fixes.

4.4 Commit Message Evaluation

Commit messages should be short, descriptive, and related to the actual changes introduced.

Commit Message	**Evaluation**
Added product management module	Clear and identifies the implemented feature.
Implemented order placement API	Clearly describes the backend functionality added.
Added inventory validation	Specifies the validation functionality introduced.
Fixed order status validation	Clearly identifies the corrected functionality.
Updated README documentation	Indicates that project documentation was modified.
Changes done	Too generic and does not describe the actual modification.
Final code	Not descriptive enough to understand what was changed.

5. Code Testing and Validation

5.1 Test Cases

ID	Test Scenario	Expected Result	Status
TC01	User registration	Account created	Pass
TC02	Valid login	User logged in	Pass
TC03	Invalid login	Error displayed	Pass
TC04	View products	Products displayed	Pass
TC05	Search product	Matching product displayed	Pass
TC06	Add product to cart	Product added	Pass
TC07	Remove product	Product removed	Pass
TC08	Place valid order	Order created	Pass
TC09	Insufficient stock	Order rejected	Pass

5.2 Order Validation

Customer Places Order

↓

Validate Order

↓

Check Product

↓

Check Inventory

↓

Create Order

↓

Update Inventory

↓

Display Confirmation

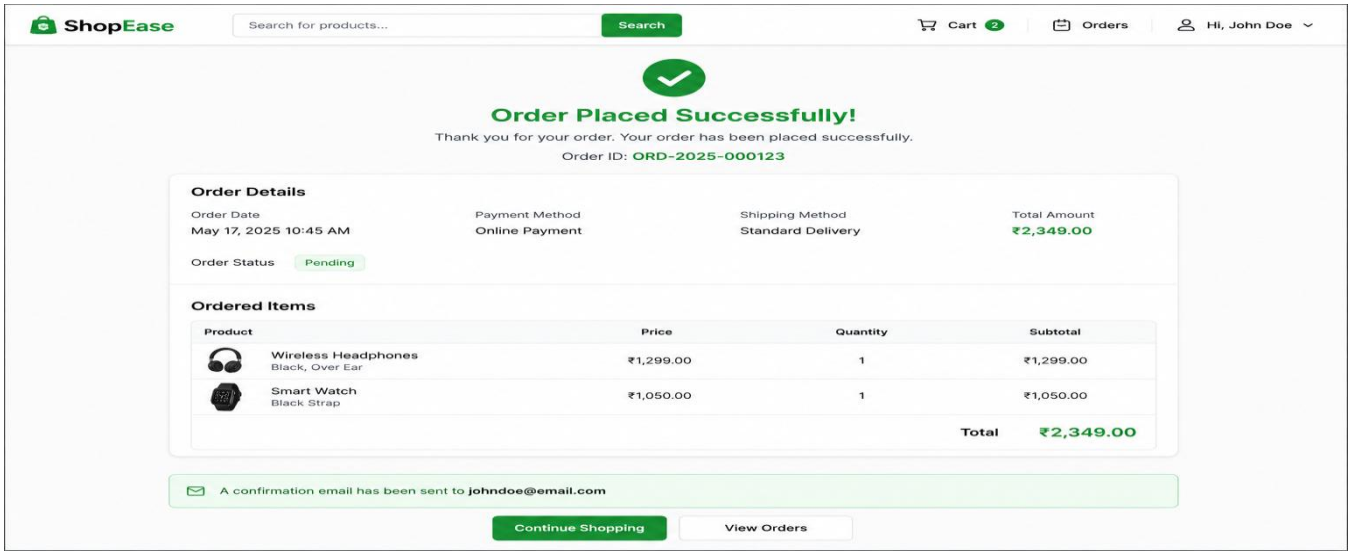


Figure 6: Successful Order Placement

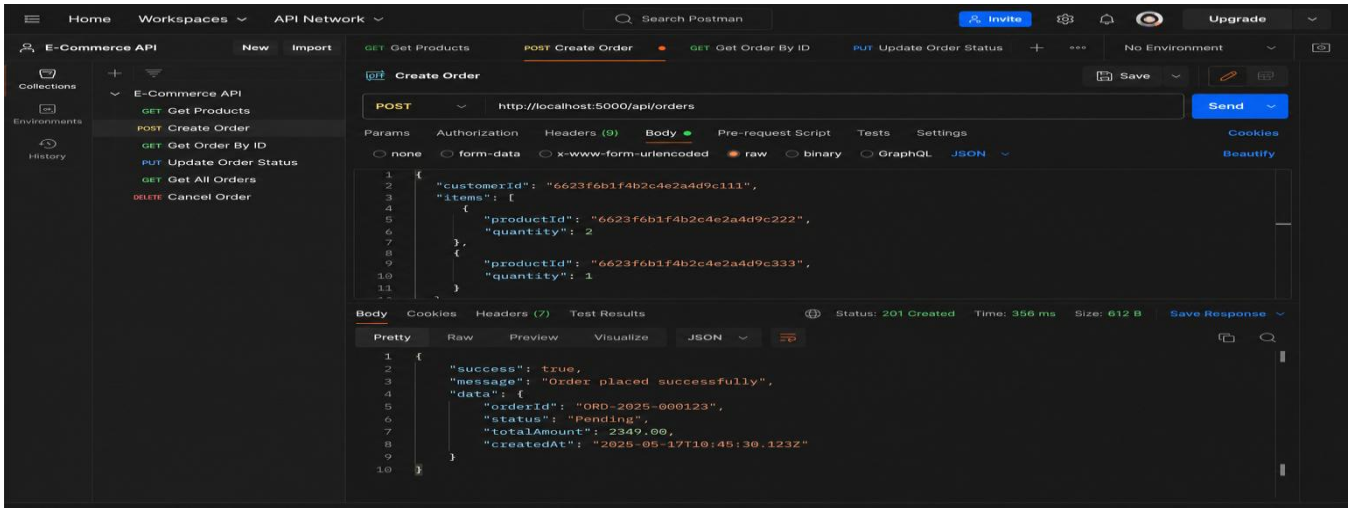


Figure 7 : Postman/API testing

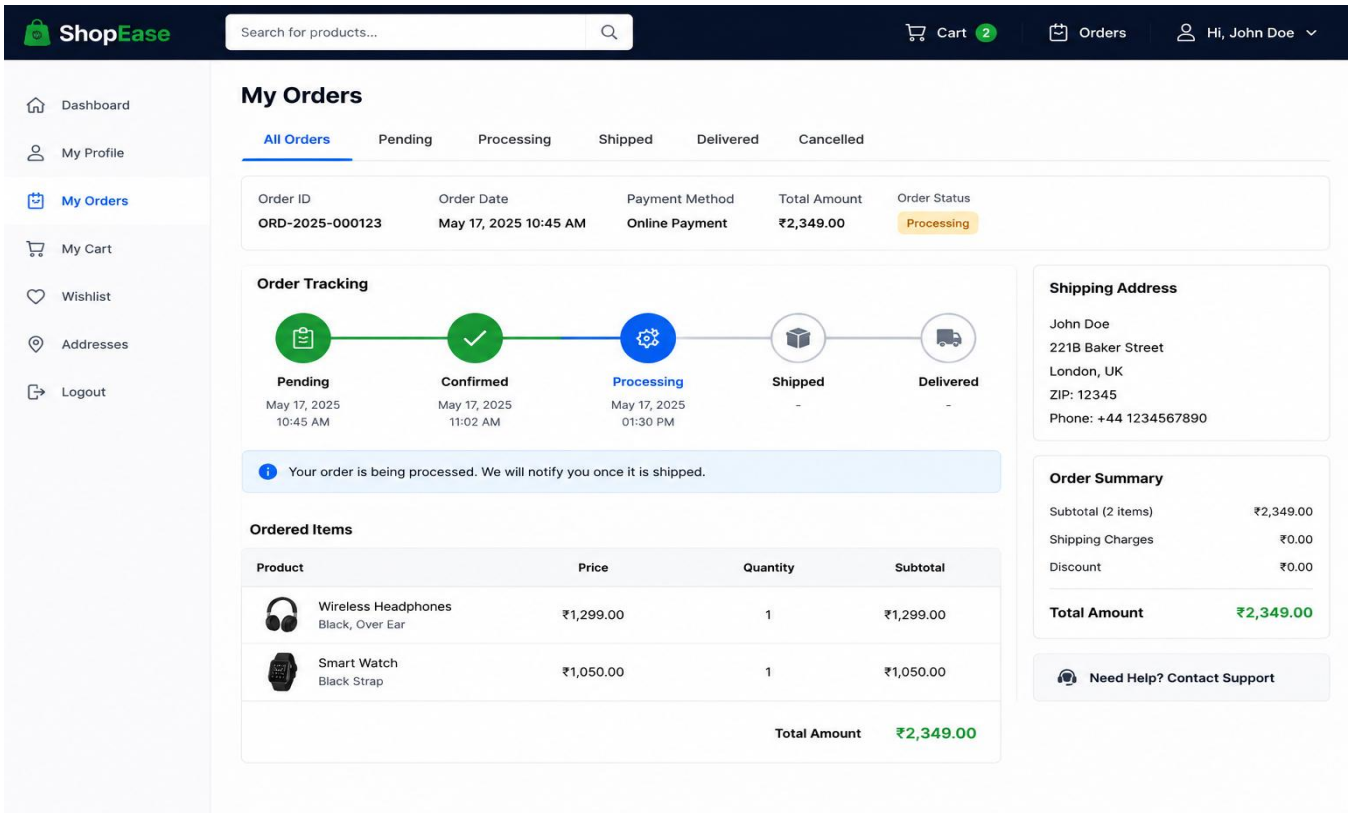


Figure 8: order Status

6. Repository Documentation

Git Repository Link: https://github.com/sana/e-commerce-order-fulfillment

The E-Commerce Order Fulfillment Platform repository documentation includes the README, installation instructions, usage guide, and project dependencies.

- **Improve Documentation:** Keep the README updated with installation steps, usage instructions, API details, and screenshots.
- **Improve Testing:** Add more unit and integration tests for order processing, inventory, authentication, and APIs.
- **Improve Error Handling:** Use centralized error-handling middleware for consistent API responses.
- **Improve Input Validation:** Validate customer, product, quantity, and order information before processing.
- **Improve Security:** Store sensitive information such as database credentials and secret keys in environment variables.

README

The README provides basic information about the project, its features, technologies used, structure.

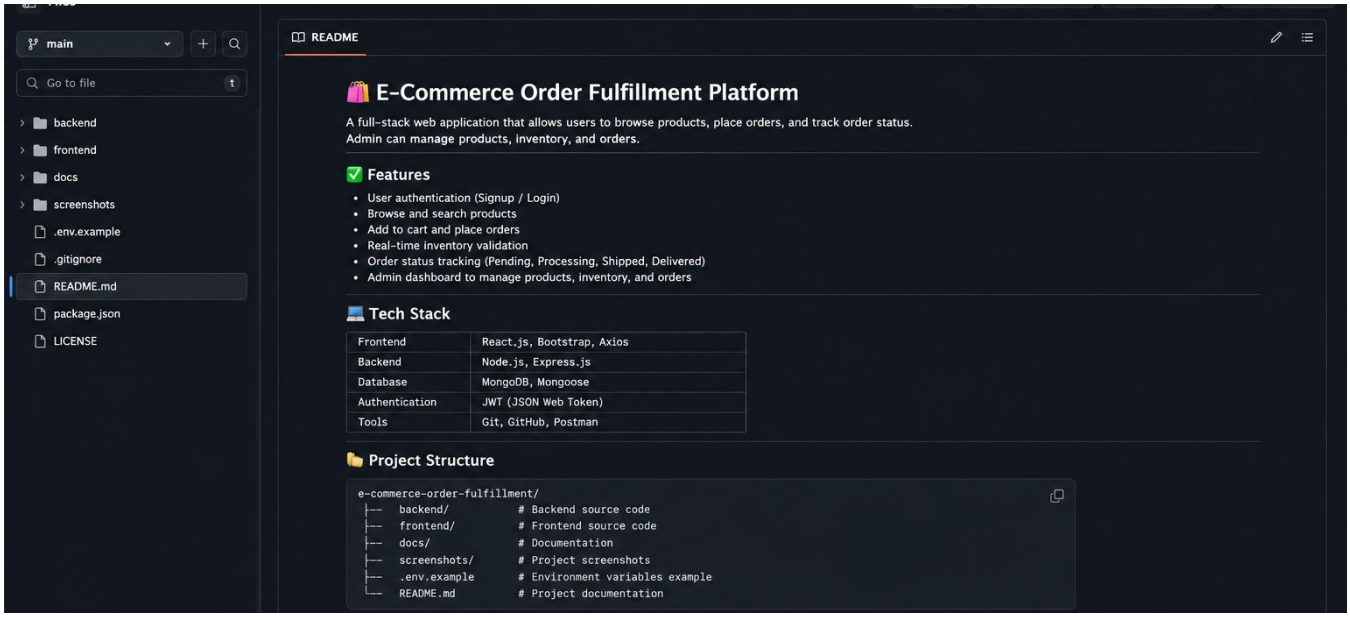


Figure 9: README Documentation

Installation Instructions

The installation section explains the basic steps required to set up and run the project, including installing dependencies and configuring the application.

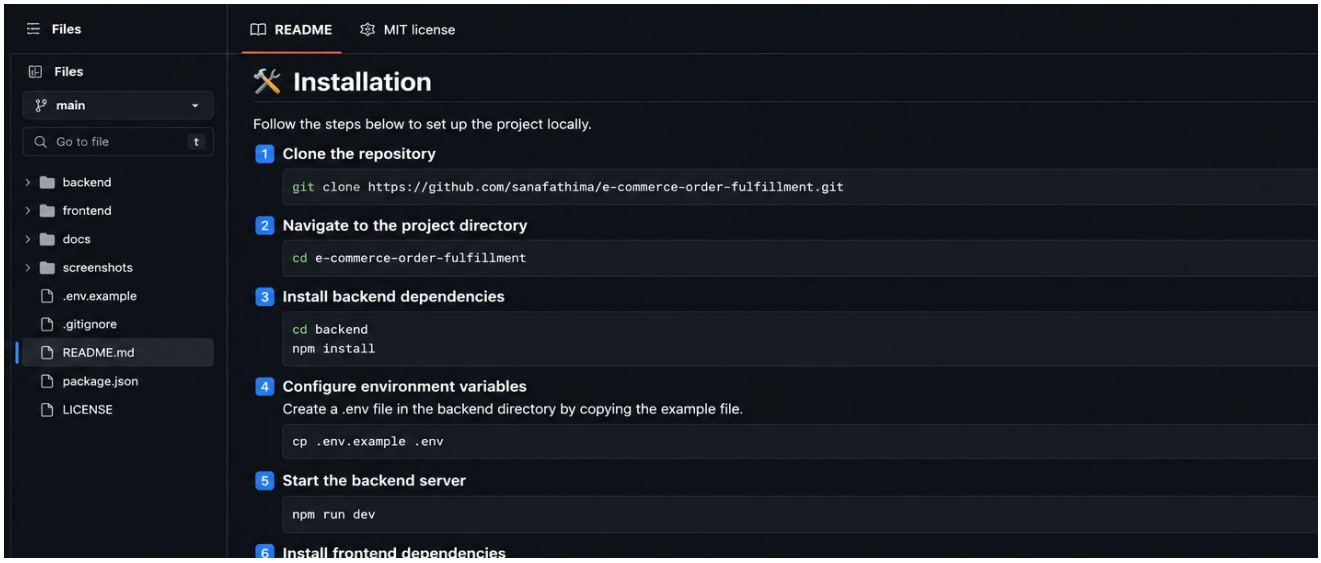


Figure 10: Installation Instructions

Usage Guide

The usage guide explains how users can access the application, browse products, add products to the cart, place orders, and track order status.

Dependencies

The project documentation lists the software packages and dependencies required to run the frontend and backend components.

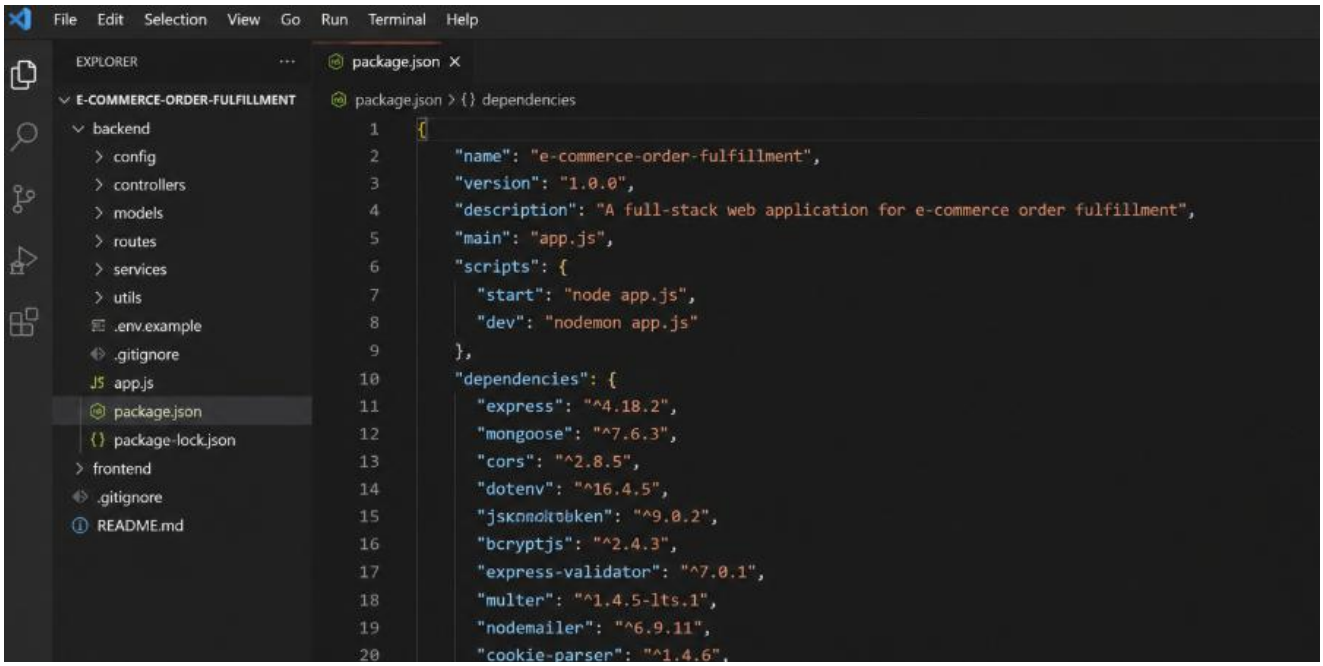


Figure 14: Project Dependencies

7. Code Review Findings and Recommendations

The overall code review of the E-Commerce Order Fulfillment Platform shows that the project has a structured repository, modular code organization, and appropriate use of version control. The main areas for improvement are code validation, error handling, testing, documentation, and repository maintenance. The assessment specifically requires findings and recommendations related to code quality, repository management, documentation, testing, and future enhancements.

7.1 Overall Findings

Area	Finding
Code Quality	The code follows a modular structure with readable names and separation of responsibilities.
Repository Management	Git provides proper tracking of code changes and supports collaborative development.
Documentation	The README provides basic project information, installation details, and dependencies.

Testing	Major functionalities such as login, product management, cart, order placement, and inventory are tested.
Maintainability	The separation of frontend and backend makes the application easier to maintain and modify.
Error Handling	Error handling can be improved through centralized error management.
Input Validation	Stronger validation should be applied to order and product-related inputs.

7.2 Recommendations

- Improve Code Quality: Use consistent coding standards, meaningful naming conventions, and reusable functions.
- Improve Repository Management: Follow a clear branching strategy and use descriptive commit messages.
- Improve Documentation: Keep the README updated with installation steps, usage instructions, API details, and screenshots.
- Improve Testing: Add more unit and integration tests for order processing, inventory, authentication, and APIs.

7.3 Conclusion

Overall, the E-Commerce Order Fulfillment Platform provides a good foundation for a maintainable and collaborative full-stack application. The project demonstrates structured code, repository organization, version control, testing, and documentation. Implementing the recommended improvements can further enhance the quality, reliability, security, usability, and maintainability of the platform.